Read-Write Lock

Read-Write Lock · Cache manifest with many readers · Build with Waqar

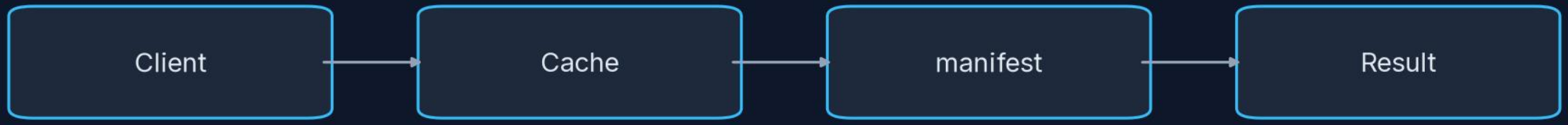
Today's Hook

- Problem: Cache manifest with many readers
- Pattern: Read-Write Lock — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Cache manifest with many readers
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Read-Write Lock: Cache manifest with many readers

Real-World Scenario

- Running example: Cache manifest with many readers
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```
package com.buildwithwaqar.patterns.concurrency.readwritelock.problem;

import java.util.*;

public final class NaiveCache {
  private String v="v1"; public String read(){ return v; } public void write(String x){ v=x; }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.concurrency.readwritelock.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

final class CachedManifest {
  private final ReadWriteLock lock=new ReentrantReadWriteLock();
  private String value="v1";
  String read(){ lock.readLock().lock(); try { return value; } finally { lock.readLock().unlock(); } }
  void write(String v){ lock.writeLock().lock(); try { value=v; } finally { lock.writeLock().unlock(); } }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.concurrency.readwritelock.problem;
import java.util.*; import java.util.concurrent.*; import java.util.concurrent.atomic.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Read-Write Lock: manifest cache ===");
    System.out.println("--- naive ---"); var n=new NaiveCache(); System.out.println(n.read()); n.write("v2"); System.out.println(n.read());
System.out.println("--- rw lock ---");
var c=new CachedManifest();
ExecutorService ex=Executors.newFixedThreadPool(4);
List<Future<String>> reads=new ArrayList<>();
for(int i=0;i<10;i++) reads.add(ex.submit(c::read));
c.write("v2");
System.out.println("final="+c.read());
ex.shutdown();
  }
}
```

Walkthrough Output

```
=== Read-Write Lock: manifest cache ===
--- naive ---
v1
v2
--- rw lock ---
final=v2
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Read-Write Lock.

Related Patterns

- Contrast Read-Write Lock with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Read-Write Lock → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.